NEXUS WORK | PPT 슬라이드 1:1 맞춤 대본 | 4조_NEXUS_WORK.PPTX 서식 통일 완료 | 디코이 취약/방어모드 실증 반영

PPT 슬라이드 연동 실전 발표 대본 (홍주형)

PPT 21페이지(우하단 22/40) 서식 및 텍스트 100% 일치 · 1부 시연(취약모드 vs 방어모드 실증) 완벽 매칭

발표자	1부 라이브 시연	2부 파트 발표	핵심 기술 키워드
홍주형 (웹·DB 디버 하드닝 / 코어)	디코이 OFF(접속허용) vs ON(403차단)	PPT 슬라이드 19~28번 + 39번 (5~7분)	RBAC 알백, IDOR 아이도어, BOLA 볼라, SSE 에스에스이

1부. 기능 구현 라이브 테스트 (팀 4인 통합 연계)

⏱ 홍주형 파트: 약 1분 (취약모드 vs 방어모드 2단계 실증)

1부 라이브 시연 화면 분할 시연

디코이(허니팟) 트리거: 취약 모드(접속 허용) vs 방어 모드(403 차단) 실증

⏱ 60~70초

🖥️ **화면 세팅:** 좌측 공격자 PowerShell 파워셸 + 우측 관제 브라우저(/admin/blocked-ips 어드민 블록트 아이피스 & /admin/audit 어드민 오딧)

🎯 **시연 핵심:** 취약모드(OFF)는 IP 차단 등록 후에도 실제 페이지 접속이 가능하지만, 방어모드(ON) 가동 즉시 403 Forbidden으로 차단되는 동작을 대조 실증!

🎙️ **구두 발표 대본 (이시헌 님에게 바통 터치받은 직후)**

"네, 이어서 웹 애플리케이션 계층의 **'디코이(허니팟) 즉시 차단 및 방어 모드 연동'** 시연을 보여드리겠습니다.

[1단계: 취약 모드(방어 모드 OFF) 상태 시연]

먼저 관제 화면 상단 스위치가 **'방어 모드 OFF 오프'** 인 취약 상태에서 공격을 진행하겠습니다.

공격자가 환경설정 파일(/.env 슬래시 닷엔브)을 찔러보는 상황을 PowerShell 파워셸에서 실행하겠습니다."

(PowerShell 파워셸 터미널에서 아래 공격 명령어 실행)

```
curl.exe -s -o NUL -w "%{http_code}`n" "http://192.168.3.10:5070/.env"
```

"터미널을 보시면 **404 Not Found 사공사 낫 파운드**가 출력되었습니다. 공격자가 눈치채지 못하도록 평범한 404 에러로 위장 응답한 것입니다.

이제 우측 브라우저의 **차단 IP 관리 화면(/admin/blocked-ips 어드민 블록트 아이피스)**을 보시면, 공격자 IP가 **'차단 중(decoy_probe:/.env)'**으로 즉시 등록되었습니다.

(강조) 하지만 보시는 것처럼 **취약 모드에서는 IP가 차단 목록에 올랐어도 실제 서버 방어가 꺼져 있기 때문에, 공격자가 포털 페이지에 그대로 접속이 가능**합니다.

[2단계: 방어 모드 ON 전환 및 403 즉시 차단 실증]

이제 관제 화면 상단의 **'방어 모드 ON 온'** 스위치를 켜겠습니다. 0.1초 만에 시스템 전체에 방어가 가동됩니다.

이 상태에서 방금 차단된 공격자 IP로 포털 페이지 재접속을 시도하겠습니다.

보시는 것처럼 **즉시 403 Forbidden 사공삼 포비든이 발생하며 페이지 접속이 전면 차단**되어 더 이상 들어가지지 않습니다!

보안 감사 로그(/admin/audit 어드민 오딧)에도 ip_blocked 아이피 블록트 판정(BLOCK 블록)으로 명확히 기록되었습니다.

원클릭으로 [차단 해제]하여 정상 복구하는 것까지 확인했습니다. 이어서 김태빈 님의 404 경로 스캔 시연으로 넘기겠습니다."

💡 **시연 필수 체크:** 관제 브라우저는 192.168.3.14(화이트리스트 보호됨)에서 띄우고, 공격 curl 컬 은 192.168.3.11(차단 가능 타깃)에서 실행하여 관제자 자가 차단을 방지합니다.

2부. 본인 파트 상세 설명 (PPT 슬라이드 1:1 연동)

⏱ 총 소요 시간: 5~7분 (슬라이드 10장 + 마무리 1장)

PPT Slide 19 우하단 20 / 40 03. 앱·DB 하드닝 (간지 — 토글로도 꺼지지 않는 권한 방어) ⏱ 30초

🖥️ **슬라이드 화면:** 03장 타이틀 · '토글로도 꺼지지 않는 권한 방어' · 쿼리부터 인가·업로드·DB 권한까지 전체 설계 책임 · 담당 홍주형

🎙️ **구두 발표 대본 (슬라이드 19번을 띄우며)**

"안녕하십니까! **NEXUS WORK** 넥서스 워크 **보안 프로젝트** 에서 **03장 앱·DB** 디비 **하드닝과 코어 보안 아키텍처** 를 총괄한 **홍주형** 입니다.
앞서 네트워크 경계와 로그 파이프라인을 보셨다면, 저는 공격자가 최종적으로 노리는 **웹 애플리케이션과 데이터베이스 자체를 근본적으로 단단하게 만드는 하드닝** 을 담당했습니다. 잘 부탁드립니다!"
(청중을 바라보며 가볍게 목례 후)
"저희 시스템의 가장 큰 특징은, 관리자가 실수로 방어 모드 스위치를 꺼서 **실제 서버 보안이 해제된 상태에서도 결코 뚫리지 않는 '절대 고정 권한 방어선'** 을 구축했다는 점입니다. 지금부터 그 설계와 실측 검증 결과를 말씀드리겠습니다."

PPT Slide 20 우하단 21 / 40 01 · 담당 범위 (앱·DB 하드닝 전체 책임 8대 설계) ⏱ 40초

🖥️ **슬라이드 화면:** 8대 설계 그리드 (1.쿼리, 2.인증, 3.세션, 4.인가, 5.출력, 6.업로드, 7.DB 권한, 8.결재·로그)

🎙️ **구두 발표 대본 (슬라이드 20번을 짚으며)**

"21페이지, 제가 총괄한 앱·DB 디비 보안 설계의 **8대 핵심 영역** 입니다.
1. **쿼리:** 모든 SQL 에스큐엘 은 파라미터 바인딩을 적용하고, 실수로 문자열 결합을 쓰지 못하도록 AST 에이에스티 정적 문법 검사로 강제했습니다.
2. **인증·세션:** 최신 암호화인 Argon2id 아르곤 투 아이디 를 적용해 5회 실패 시 15분 잠금 및 계정 열거 방지를 구현했고, 세션은 HttpOnly 에이치티티피 온리 와 SameSite 세임 사이트 로 탈취를 막았습니다.
3. **인가:** 로그인 세션만 믿는 것이 아니라, 매 요청마다 DB 디비 에서 본인 소유 범위를 실시간 재검증합니다.
4. **DB 디비 권한 분리:** 일반 테이블은 쓰기 계정을 쓰지만, 기밀 문서나 급여 테이블은 웹 서버가 오직 **읽기 전용**으로만 접근하도록 물리적으로 격리했습니다."

PPT Slide 21 우하단 22 / 40 02 · 개념 (왜 데모 스위치와 무관한가 — 디코이 vs 권한 방어 대조) 45초

🖥️ **슬라이드 화면 텍스트 (서식 100% 일치):**
• "R1·R2·R8(디코이 IP 차단)은 SECURITY_MODE 토글로 제어됩니다. 취약 모드(OFF)에서는 IP 차단 후에도 접속되지만, 방어 모드(ON)에서는 403으로 차단됩니다. 반면 R9(비인가 관리자 경로)·R10(IDOR·기밀 접근)은 **관리자 권한 체크·역할 체크·리소스 범위 체크가 토글 변수를 아예 참조하지 않는, 서버 측에 항상 고정된 검사**이기 때문에 데모 모드로도 끌 수 없습니다."
• 하단 3대 기둥: 🪪 역할 기반 접근 제어(auth.py) · 🛡️ 관리자 전용 라우트 가드(admin.py) · 🗂️ 리소스 범위 검증(hr.py)

🎙️ **구두 발표 대본 (슬라이드 21번 핵심 개념 설명)**

"22페이지는 시스템의 핵심 개념인 **'왜 방어 모드 스위치와 무관하게 뚫리지 않는가'** 입니다.
슬라이드 상단 요약문처럼, R1 알원 브루트포스나 R8 알에이트 디코이 IP 차단은 토글 스위치로 제어됩니다. 1부 시연에서 직접 보셨듯이 **취약 모드(OFF 오프)에서는 IP가 차단 목록에 올라도 페이지 접속이 유지되지만, 방어 모드(ON 온)에서는 즉시 403 사공삼으로 전면 차단** 됩니다.

하지만 **제가 구현한 R9 알나인 (관리자 권한)과 R10 알텐 (IDOR 아이도어 ·기밀 통제)은 스위치를 OFF 오프로 꺼도 절대로 뚫리지 않습니다.**
그 이유는 auth.py의 역할 검사, admin.py의 관리자 가드, hr .py의 리소스 소유권 검사가 토글 변수를 아예 참조하지 않고, **서버 백엔드 로직 자체에 '항상 고정된 불변의 룰'로 박혀 있기 때문** 입니다."

💡 **강조 포인트:** 슬라이드의 보라색 강조 문구(관리자 권한 체크·역할 체크·리소스 범위 체크가 고정된 검사)를 손으로 짚으며 R9/R10의 불변성을 확신 있게 발언합니다.

PPT Slide 22 우하단 23 / 40 03 · 라이브 검증 (test 계정으로 실제 검증한 결과표) 40초

🖥️ **슬라이드 화면:** test 계정(인사카드 22번) 실측 결과표 (/hr/cards/22 ➔ 200 OK / 타인카드 ➔ 403 / 기밀문서 ➔ 403 / 관리자화면 ➔ 403)

🎙️ **구두 발표 대본 (슬라이드 22번 실측 데이터 제시)**

"23페이지는 일반사원 test 테스트 계정으로 실제 라이브 환경에서 검증한 실측 결과표입니다.
보시는 것처럼 본인의 인사카드인 22번은 **200 OK** 이백 오케이 로 정상 조회되지만,
- URL 유알엘 번호만 1번으로 조작해 타인 카드를 훔쳐보려는 IDOR 아이도어 공격,
- /confidential 컨피덴셜 기밀 문서함 접근,
- 그리고 /admin/* 어드민 관리자 메뉴 접근은 **예외 없이 403 Forbidden** 사공삼 포비든 으로 즉시 차단되었습니다.
방어 모드를 ON 온 으로 두든 OFF 오프 로 풀든 결과는 100% 동일하게 철통 방어됨을 확인했습니다."

PPT Slide 23 우하단 24 / 40

04 · 실측 캡처 (실제 화면으로 확인한 차단)

30초

슬라이드 화면: 좌측: GET /hr/cards/1 타인 인사카드 차단 캡처 · 우측: GET /admin/users 관리자 화면 차단 캡처

구두 발표 대본 (슬라이드 23번 캡처 증적 제시)

"24페이지는 실제 브라우저에서 test 테스트 계정으로 로그인해 차단된 화면을 직접 캡처한 실측 증적입니다.
좌측은 주소창 번호만 1번으로 바꿔 타인 인사카드를 열람하려다 403 차단된 화면이며,
우측은 일반 사원이 관리자 콘솔인 /admin/users 어드민 유저스 에 무단 접근하려다 차단된 화면입니다.
클라이언트 UI가 아닌 백엔드 커널 레벨에서 거부하기 때문에 화면 조작으로는 결코 우회할 수 없습니다."

PPT Slide 24 우하단 25 / 40

05 · 탐지 연동 (Graylog R9 · R10 — 거부 이벤트로 남는 흔적)

40초

슬라이드 화면: R9(insufficient_role) & R10(out_of_scope) 감사 로그 콘솔 흐름 · Graylog 실시간 추적 다이어그램

구두 발표 대본 (슬라이드 24번 관제 연동 설명)

"25페이지는 이 차단 시도가 어떻게 상위 관제 센터로 연동되는지를 보여줍니다.
비인가 접근이 차단되는 즉시, 보안 감사 로그(/admin/audit 어드민 오딧)에 authz_deny 어스지 디나이 이벤트가 실시간 기록됩니다.
관리자 무단 접근은 insufficient_role 인서피션트 롤 , 타인 카드 조회는 out_of_scope 아웃 오브 스코프 라는 명확한 사유와 함께 요청자 계정과 IP 아이피 가 남으며,
이 이벤트는 Graylog 그레이로그 로 실시간 파이프라인 전송되어 SOC 에스오씨 관제 요원이 침해 시도를 즉시 추적할 수 있습니다."

PPT Slide 25 우하단 26 / 40 **06 · 결재 무결성 (5단계 결재 BOLA 차단 & 지출결의서 서버 재검산)** 50초

🖥️ **슬라이드 화면:** 5단계 순차 결재 라인 · BOLA 차단(not_your_turn) · 지출결의서 수량×단가 서버 강제 재검산 · app/approvals.py 코드

🎙️ **구두 발표 대본 (⭐ 가장 인상 깊었던 백엔드 코드)**

"26페이지는 제 작업 중 가장 인상 깊었던 백엔드 코드인 app/approvals.py 앱 슬래시 어프루벌스 닷 파이 의 **'전자결재 BOLA 볼라 방어와 서버 강제 재검산'** 입니다.

첫째, **순번 우회(BOLA 볼라) 차단:** 기안부터 최종승인까지 5단계 순차 결재를 강제하여, 본인 차례가 아니거나 중간 승인자를 건너뛰면 403 'not_your_turn' 낫 유어 턴 으로 거부합니다.
둘째, **지출결의서 금액 위조 방어:** 공격자가 브라우저 개발자 도구에서 품의 총금액을 100만 원에서 1만 원으로 위조해 POST 포스트 요청을 전송하더라도, 서버는 클라이언트의 합계를 믿지 않습니다.
서버 CPU 씨피유가 직접 품목별 수량 × 단가를 강제 재연산 하여 불일치 시 **400 Bad Request** **사백 배드 리퀘스트**('tamper_detected'` 탬퍼 디텍티드)로 튕겨내고 위조 시도 자체를 감사 로그에 박제합니다."

PPT Slide 26 우하단 27 / 40 07 · 코어 개선 (실시간 보안 모드 동기화 버그 수정) 45초

🖥️ 슬라이드 화면: 버그(SSE 이벤트 핸들러 누락) ➔ 수정(SSE 브로드캐스트 + 0.1초 갱신 + 3초 폴링/탭 복귀 이중 백업) · audit-modal.js 코드

🎙️ 구두 발표 대본 (슬라이드 26번 문제 해결 1)

"27페이지는 개발 중 끝까지 추적해 해결한 실시간 보안 모드 동기화 버그 입니다.
관리자가 방어 모드를 토글해도, 다른 자리에 있는 관제 모니터는 새로고침 전까지 이전 상태로 멈춰 있던 문제가 있었습니다. 원인을 분석해 보니 SSE 에스에스이 스트림 이벤트 핸들러가 누락되어 있었습니다.

➔ 해결: app/audit.py 앱 슬래시 오딧 닷 파이 에서 모드 전환 즉시 SSE 에스에스이 큐로 이벤트를 쏴주고, audit-modal.js 오딧 모달 닷 제이에스 가 0.1초 만에 배지와 버튼을 즉각 갱신 하도록 고쳤습니다. 여기에 3초 폴링과 탭 복귀 자동 동기화까지 3중 백업을 완성했습니다."

PPT Slide 27 | 우하단 28 / 40

08 · 성능 최적화 (AI 판정 4중 캐스케이드로 15~30초 → 0.5~1.0초)

50초

🖥️ 슬라이드 화면: 4중 캐스케이드(1.Groq LPU ➔ 2.NVIDIA NIM ➔ 3.Docker Ollama ➔ 4.Builtin Rule) · 98% 속도 단축 · 251개 E2E 테스트 통과

🎙️ 구두 발표 대본 (슬라이드 27번 문제 해결 2)

"28페이지는 AI 에이아이 분석 병목을 해소한 성능 최적화 결과입니다.
기존 로컬 Docker Ollama 도커 올라마 단일 의존 구조에서는 침해 분석에 15초에서 30초가 걸려 앞단의 방화벽과 프록시가 타임아웃 504 오공사로 터지는 병목이 발생했습니다.

➔ 해결: 클라우드와 온프레미스를 결합한 'AI 에이아이 4중 캐스케이드' 파이프라인을 구축했습니다.
1순위 초고속 Groq Cloud LPU 그록 클라우드 엘피유 (0.5~1초) 최우선 라우팅 ➔ 장애 시 2순위 NVIDIA NIM 엔비디아 님 ➔ 인터넷 단절 시 3순위 Docker Ollama 도커 올라마 ➔ 비상시 4순위 내장 룰 엔진(0.01초)으로 자동 승계됩니다.
그 결과 속도를 98% 단축 시켰고, 251개 E2E 이투이 자동화 테스트를 42초 만에 완주하며 기존 기능 손상 없는 Zero-Regression 제로 리그레션을 달성했습니다."

PPT Slide 28 | 우하단 29 / 40

09 · 다음 계획 (문제 해결과 다음 계획)

40초

🖥️ 슬라이드 화면: 캡처 스크립트 404 해결 · Windows 파일 핸들 잠금 안정화 · 다음 계획(리허설 연습, 화면 분할 세팅, 향후 개선)

🎙️ 구두 발표 대본 (슬라이드 28번 향후 계획 제시)

"29페이지는 제작 과정의 문제 해결 경험과 향후 고도화 계획입니다.
자동 캡처 스크립트가 비활성 라우트를 찍어 404가 나오던 문제를 실제 동작 콘솔로 전부 재검증해 교체했고, Windows 윈도우의 파일 잠금 오류도 예외 처리로 안정화했습니다.

향후 개선 방향:
1. 중앙 인가 엔진(OPA 오파) 도입: 현재 파이썬 데코레이터 기반 인가를 Open Policy Agent 오픈 폴리시 에이전트로 모듈화하여, 근무 시간 외 차단 등 동적 정책을 코드 수정 없이 중앙 배포하도록 발전시키겠습니다.
2. FIDO2 파이도 투 패스키 2차 인증(MFA 엠에프에이): 임원 결재나 기밀 문서 열람 시 피싱이 불가능한 하드웨어 패스키 인증을 강제해 계정 탈취 위협을 원천 봉쇄하겠습니다."

PPT Slide 39 우하단 40 / 40 마무리 (4명, 4개 방어선, 하나의 체계) & 예상 Q&A 3선 1분 30초

🖥️ 슬라이드 화면: 4명 4개 방어선 통합 다이어그램 (이시헌 pfSense → 김태빈 Graylog/AI-XDR → 홍주형 앱·DB 하드닝 → 진혜정 라이브 검증)

🎙️ 구두 발표 대본 (팀 마무리 총평)

"마지막 40페이지, 팀 전체 마무리입니다.
네트워크 경계와 로그 파이프라인이 뚫리더라도, 제가 구축한 앱·DB 디비 하드닝 계층이 최후의 보루로서 '데이터 변조 불가'와 '권한 우회 불가'를 철저히 지켜내어 전체 팀의 유기적인 다중 방어 체계를 완성했습니다.
질문을 받기에 앞서, 심사위원분들께서 가장 궁금해하실 만한 핵심 질문 3가지를 미리 답변드리겠습니다."

❓ Q1 큐원. 방어 모드 버튼을 OFF로 누르면 실제 보안이 꺼지나요, 아니면 화면만 바뀌는 더미(데모) 모드인가요?

🎙️ 답변: "단순한 화면 더미가 아니라, 실제 실행 중인 파이썬 서버의 메모리(current_app.config 커런트 앱 닷 컨피그)와 환경변수를 조작해 실제 방어를 해제하는 진짜 런타임 스위치입니다. 1부 시연에서 보셨듯이 실제로 OFF 오프 상태에서는 디코이 IP 차단이 등록되어도 실제 접속은 통과되고, 비밀번호 5회 실패 잠금이 999999회로 풀립니다. 하지만 제가 구현한 R9 알나인(관리자 권한), R10 알텐(IDOR 아이도어), 결재 무결성은 이 스위치를 OFF 오프로 꺼도 결코 꺼지지 않는 '절대 고정 방어선'으로 심어두었기 때문에 403 사공삼 차단이 유지됩니다."

❓ Q2 큐투. 전자결재 금액 위조(BOLA 볼라/변조)를 서버가 어떻게 감지하고 막았다는 건가요?

🎙️ 답변: "공격자가 브라우저 개발자 도구에서 결재 총금액을 100만 원에서 1만 원으로 조작해 POST 포스트 요청을 보낼 수 있습니다. 저희 백엔드(app/approvals.py 앱 슬래시 어프루벌스 닷 파이)는 브라우저가 보낸 총금액 필드를 아예 신뢰하지 않고, 서버 CPU 씨피유가 직접 각 품목의 수량과 단가를 가져와 재연산합니다. 클라이언트 금액과 1원이라도 다르면 즉시 400 사백 'tamper_detected' 탬퍼 디텍티드 에러를 내며 거래를 거부하고 감사 로그를 남깁니다."

❓ Q3 큐쓰리. AI 에이아이 4중 캐스케이드에서 1순위 Groq 그록 클라우드로 보낼 때 회사 기밀 로그가 유출될 위험은 없나요?

🎙️ 답변: "매우 중요한 지적이십니다. 이를 위해 로그 전송 전 정규식을 통해 비밀번호, 주민번호, 개인 토큰 등 민감 정보를 철저히 마스킹(가림 처리)하여 전송합니다. 또한 만약 외부 인터넷 연결이 완전히 차단된 폐쇄망 군·공공기관 환경일 경우, 환경 변수 변경 한 줄로 외부 클라우드를 건너뛰고 3순위 사내 Docker Ollama 도커 올라마나 4순위 내장 룰 엔진으로만 안전하게 작동하도록 설계되어 있습니다."

(답변 후) "혹시 추가적인 질문이 있으시면 말씀해 주시기 바랍니다. 감사합니다!"